



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

Лабораторная работа № 1
ПО КУРСУ
«Теория игр в системах защиты информации»
на тему «Матричные игры с нулевой суммой.
Смешанные стратегии»

Студент

ИУ8-22М
(Группа)

Е. В. Умеренко
(И. О. Фамилия)

Преподаватель:

П. В. Захаров
(И.О. Фамилия)

Цель работы

Изучить постановку антагонистической игры двух лиц в нормальной форме; найти решение игры за обоих игроков в смешанных стратегиях (стратегическую седловую точку).

Постановка задачи

Для игры, заданной матрицей стратегий требуется найти оптимальные смешанные стратегии обоих игроков, сведя матричную игру к задаче ЛП (прямой для одного игрока и двойственной для другого). Задачи ЛП следует решать симплекс-методом, приведя начальные, промежуточные и конечные симплекс-таблицы.

Ход работы

Вариант 23. Заданная матрица стратегий показана на рисунке 1.

$$\begin{bmatrix} 7 & 19 & 1 & 19 & 8 \\ 7 & 18 & 5 & 2 & 6 \\ 15 & 3 & 16 & 19 & 4 \\ 5 & 12 & 19 & 14 & 18 \end{bmatrix}$$

Рисунок 1 – Матрица стратегий

Матричная игра сведена к задаче ЛП и решена симплекс-методом. Начальные, промежуточные и конечные симплекс-таблицы показаны для игрока А (раздел 1) и для игрока Б (раздел 2).

Решение для игрока А

Результат вывода программы в виде симплекс-таблиц для игрока А представлен в листинге ниже:

```
=====
РЕШЕНИЕ ДЛЯ ИГРОКА А (МИНИМИЗАЦИЯ)
=====

НАЧАЛЬНАЯ СИМПЛЕКС-ТАБЛИЦА
=====
|  С  |  1  |  1  |  1  |  1  |  0  |  0  |  0  |  0  |  0  |  0  |  0  |
| базис | u1 | u2 | u3 | u4 | s1 | s2 | s3 | s4 | s5 | b  |
-----
|  s1  | -7.0000  -7.0000  -15.0000  -5.0000  1.0000  0.0000  0.0000  0.0000  0.0000  -1.0000 |
|  s2  | -19.0000 -18.0000  -3.0000 -12.0000  0.0000  1.0000  0.0000  0.0000  0.0000  -1.0000 |
|  s3  | -1.0000  -5.0000 -16.0000 -19.0000  0.0000  0.0000  1.0000  0.0000  0.0000  -1.0000 |
|  s4  | -19.0000 -2.0000 -19.0000 -14.0000  0.0000  0.0000  0.0000  1.0000  0.0000  -1.0000 |
|  s5  | -8.0000  -6.0000  -4.0000 -18.0000  0.0000  0.0000  0.0000  0.0000  1.0000  -1.0000 |
| Z/W  |  1.0000  1.0000  1.0000  1.0000  0.0000  0.0000  0.0000  0.0000  0.0000  0.0000 |
=====

--- Итерация 1 (Фаза 1) ---
Отрицательный b в строке 0 = -1.0000
Ведущий столбец: 0, элемент = -7.0000

ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ 1
=====
```

	с		1		1		1		1		0		0		0		0		0		0		0	
	базис		u1		u2		u3		u4		s1		s2		s3		s4		s5		b			

	u1		1.0000		1.0000		2.1429		0.7143		-0.1429		-0.0000		-0.0000		-0.0000		-0.0000		-0.0000		0.1429	
	s2		0.0000		1.0000		37.7143		1.5714		-2.7143		1.0000		0.0000		0.0000		0.0000		0.0000		1.7143	
	s3		0.0000		-4.0000		-13.8571		-18.2857		-0.1429		0.0000		1.0000		0.0000		0.0000		0.0000		-0.8571	
	s4		0.0000		17.0000		21.7143		-0.4286		-2.7143		0.0000		0.0000		1.0000		0.0000		0.0000		1.7143	
	s5		0.0000		2.0000		13.1429		-12.2857		-1.1429		0.0000		0.0000		0.0000		0.0000		1.0000		0.1429	
	Z/W		0.0000		0.0000		-1.1429		0.2857		0.1429		0.0000		0.0000		0.0000		0.0000		0.0000		-0.1429	

--- Итерация 2 (Фаза 1) ---

Отрицательный b в строке 2 = -0.8571

Ведущий столбец: 1, элемент = -4.0000

ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ 2

	с		1		1		1		1		0		0		0		0		0		0		0	
	базис		u1		u2		u3		u4		s1		s2		s3		s4		s5		b			

	u1		1.0000		0.0000		-1.3214		-3.8571		-0.1786		0.0000		0.2500		0.0000		0.0000		0.0000		-0.0714	
	s2		0.0000		0.0000		34.2500		-3.0000		-2.7500		1.0000		0.2500		0.0000		0.0000		0.0000		1.5000	
	u2		-0.0000		1.0000		3.4643		4.5714		0.0357		-0.0000		-0.2500		-0.0000		-0.0000		-0.0000		0.2143	
	s4		0.0000		0.0000		-37.1786		-78.1429		-3.3214		0.0000		4.2500		1.0000		0.0000		0.0000		-1.9286	
	s5		0.0000		0.0000		6.2143		-21.4286		-1.2143		0.0000		0.5000		0.0000		1.0000		1.0000		-0.2857	
	Z/W		0.0000		0.0000		-1.1429		0.2857		0.1429		0.0000		0.0000		0.0000		0.0000		0.0000		-0.1429	

--- Итерация 3 (Фаза 1) ---

Отрицательный b в строке 3 = -1.9286

Ведущий столбец: 2, элемент = -37.1786

ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ 3

С	1	1	1	1	0	0	0	0	0	0	0
базис	u1	u2	u3	u4	s1	s2	s3	s4	s5	b	
u1	1.0000	0.0000	0.0000	-1.0797	-0.0605	0.0000	0.0989	-0.0355	0.0000	-0.0029	
s2	0.0000	0.0000	0.0000	-74.9875	-5.8098	1.0000	4.1652	0.9212	0.0000	-0.2767	
u2	0.0000	1.0000	0.0000	-2.7099	-0.2738	0.0000	0.1460	0.0932	0.0000	0.0346	
u3	-0.0000	-0.0000	1.0000	2.1018	0.0893	-0.0000	-0.1143	-0.0269	-0.0000	0.0519	
s5	0.0000	0.0000	0.0000	-34.4899	-1.7695	0.0000	1.2104	0.1671	1.0000	-0.6081	
Z/W	0.0000	0.0000	0.0000	2.6878	0.2450	0.0000	-0.1306	-0.0307	0.0000	-0.0836	

--- Итерация 4 (Фаза 1) ---

Отрицательный b в строке 4 = -0.6081

Ведущий столбец: 3, элемент = -34.4899

ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ 4

С	1	1	1	1	0	0	0	0	0	0	0
базис	u1	u2	u3	u4	s1	s2	s3	s4	s5	b	

	u1		1.0000	0.0000	0.0000	0.0000	-0.0051	0.0000	0.0611	-0.0408	-0.0313	0.0162	
	s2		0.0000	0.0000	0.0000	0.0000	-1.9627	1.0000	1.5336	0.5578	-2.1742	1.0454	
	u2		0.0000	1.0000	0.0000	0.0000	-0.1347	0.0000	0.0509	0.0800	-0.0786	0.0824	
	u3		0.0000	0.0000	1.0000	0.0000	-0.0185	0.0000	-0.0406	-0.0167	0.0609	0.0148	
	u4		-0.0000	-0.0000	-0.0000	1.0000	0.0513	-0.0000	-0.0351	-0.0048	-0.0290	0.0176	
	Z/W		0.0000	0.0000	0.0000	0.0000	0.1071	0.0000	-0.0363	-0.0177	0.0779	-0.1310	
=====													

--- Итерация 5 (Фаза 2) ---

Дельты: ['0.0000', '0.0000', '0.0000', '0.0000', '-0.1071', '0.0000', '0.0363', '0.0177', '-0.0779']

Ведущий столбец: 6, дельта = 0.0363

Ведущая строка: 0, отношение = 0.2646

ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ 5

	С		1		1		1		1		0		0		0		0		0		0	
	базис		u1		u2		u3		u4		s1		s2		s3		s4		s5		b	

	s3		16.3796		0.0000		0.0000		0.0000		-0.0839		0.0000		1.0000		-0.6679		-0.5128		0.2646	
	s2		-25.1204		0.0000		0.0000		0.0000		-1.8339		1.0000		0.0000		1.5821		-1.3878		0.6396	
	u2		-0.8339		1.0000		0.0000		0.0000		-0.1305		0.0000		0.0000		0.1141		-0.0525		0.0689	
	u3		0.6642		0.0000		1.0000		0.0000		-0.0219		0.0000		0.0000		-0.0438		0.0401		0.0255	
	u4		0.5748		0.0000		0.0000		1.0000		0.0484		0.0000		0.0000		-0.0283		-0.0470		0.0269	
	Z/W		0.5949		0.0000		0.0000		0.0000		0.1040		0.0000		0.0000		-0.0420		0.0593		-0.1214	
=====																						

--- Итерация 6 (Фаза 2) ---

Дельты: ['-0.5949', '0.0000', '0.0000', '0.0000', '-0.1040', '0.0000', '0.0000', '0.0420', '-0.0593']

Ведущий столбец: 7, дельта = 0.0420

Ведущая строка: 1, отношение = 0.4043

ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ 6

	с	1	1	1	1	0	0	0	0	0	0	
	базис	u1	u2	u3	u4	s1	s2	s3	s4	s5	b	
	s3	5.7751	0.0000	0.0000	0.0000	-0.8581	0.4221	1.0000	0.0000	-1.0986	0.5346	
	s4	-15.8777	0.0000	0.0000	0.0000	-1.1592	0.6321	0.0000	1.0000	-0.8772	0.4043	
	u2	0.9769	1.0000	0.0000	0.0000	0.0017	-0.0721	0.0000	0.0000	0.0476	0.0228	
	u3	-0.0311	0.0000	1.0000	0.0000	-0.0727	0.0277	0.0000	0.0000	0.0017	0.0433	
	u4	0.1257	0.0000	0.0000	1.0000	0.0156	0.0179	0.0000	0.0000	-0.0718	0.0384	
	Z/W	-0.0715	0.0000	0.0000	0.0000	0.0554	0.0265	0.0000	0.0000	0.0225	-0.1044	

--- Итерация 7 (Фаза 2) ---

Дельты: ['0.0715', '0.0000', '0.0000', '0.0000', '-0.0554', '-0.0265', '0.0000', '0.0000', '-0.0225']

Ведущий столбец: 0, дельта = 0.0715

Ведущая строка: 2, отношение = 0.0233

ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ 7

	с	1	1	1	1	0	0	0	0	0	0	
--	---	---	---	---	---	---	---	---	---	---	---	--

базис	u1	u2	u3	u4	s1	s2	s3	s4	s5	b	

s3	0.0000	-5.9115	0.0000	0.0000	-0.8684	0.8483	1.0000	0.0000	-1.3799	0.3999	
s4	0.0000	16.2527	0.0000	0.0000	-1.1311	-0.5396	0.0000	1.0000	-0.1039	0.7745	
u1	1.0000	1.0236	0.0000	0.0000	0.0018	-0.0738	0.0000	0.0000	0.0487	0.0233	
u3	0.0000	0.0319	1.0000	0.0000	-0.0726	0.0254	0.0000	0.0000	0.0032	0.0440	
u4	0.0000	-0.1287	0.0000	1.0000	0.0153	0.0272	0.0000	0.0000	-0.0779	0.0354	
Z/W	0.0000	0.0732	0.0000	0.0000	0.0555	0.0213	0.0000	0.0000	0.0260	-0.1027	
=====											

РЕЗУЛЬТАТ ДЛЯ ИГРОКА А

Минимальное значение W = -0.102715

Цена игры v = -9.735632

Оптимальная стратегия А:

x1 = 0.023318

x2 = 0.000000

x3 = 0.043979

x4 = 0.035419

Решение для игрока Б

Результат вывода программы в виде симплекс-таблиц для игрока Б представлен в листинге ниже:

РЕШЕНИЕ ДЛЯ ИГРОКА В (МАКСИМИЗАЦИЯ)

НАЧАЛЬНАЯ СИМПЛЕКС-ТАБЛИЦА

С	-1	-1	-1	-1	-1	0	0	0	0	0	
базис	t1	t2	t3	t4	t5	r1	r2	r3	r4	b	
r1	7.0000	19.0000	1.0000	19.0000	8.0000	1.0000	0.0000	0.0000	0.0000	1.0000	
r2	7.0000	18.0000	5.0000	2.0000	6.0000	0.0000	1.0000	0.0000	0.0000	1.0000	
r3	15.0000	3.0000	16.0000	19.0000	4.0000	0.0000	0.0000	1.0000	0.0000	1.0000	
r4	5.0000	12.0000	19.0000	14.0000	18.0000	0.0000	0.0000	0.0000	1.0000	1.0000	
Z/W	-1.0000	-1.0000	-1.0000	-1.0000	-1.0000	0.0000	0.0000	0.0000	0.0000	0.0000	

--- Итерация 1 (Фаза 2) ---

Дельты: ['1.0000', '1.0000', '1.0000', '1.0000', '1.0000', '0.0000', '0.0000', '0.0000', '0.0000']

Ведущий столбец: 0, дельта = 1.0000

Ведущая строка: 2, отношение = 0.0667

ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ 1

С	-1	-1	-1	-1	-1	0	0	0	0	0	
базис	t1	t2	t3	t4	t5	r1	r2	r3	r4	b	
r1	0.0000	17.6000	-6.4667	10.1333	6.1333	1.0000	0.0000	-0.4667	0.0000	0.5333	
r2	0.0000	16.6000	-2.4667	-6.8667	4.1333	0.0000	1.0000	-0.4667	0.0000	0.5333	
t1	1.0000	0.2000	1.0667	1.2667	0.2667	0.0000	0.0000	0.0667	0.0000	0.0667	
r4	0.0000	11.0000	13.6667	7.6667	16.6667	0.0000	0.0000	-0.3333	1.0000	0.6667	
Z/W	0.0000	-0.8000	0.0667	0.2667	-0.7333	0.0000	0.0000	0.0667	0.0000	0.0667	

=====

--- Итерация 2 (Фаза 2) ---

Дельты: ['0.0000', '0.8000', '-0.0667', '-0.2667', '0.7333', '0.0000', '0.0000', '-0.0667', '0.0000']

Ведущий столбец: 1, дельта = 0.8000

Ведущая строка: 0, отношение = 0.0303

=====

ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ 2

=====

	С		-1		-1		-1		-1		-1		0		0		0		0		0		0	
	базис		t1		t2		t3		t4		t5		r1		r2		r3		r4		b			

	t2		0.0000		1.0000		-0.3674		0.5758		0.3485		0.0568		0.0000		-0.0265		0.0000		0.0303	
	r2		0.0000		0.0000		3.6326		-16.4242		-1.6515		-0.9432		1.0000		-0.0265		0.0000		0.0303	
	t1		1.0000		0.0000		1.1402		1.1515		0.1970		-0.0114		0.0000		0.0720		0.0000		0.0606	
	r4		0.0000		0.0000		17.7083		1.3333		12.8333		-0.6250		0.0000		-0.0417		1.0000		0.3333	
	Z/W		0.0000		0.0000		-0.2273		0.7273		-0.4545		0.0455		0.0000		0.0455		0.0000		0.0909	

=====

--- Итерация 3 (Фаза 2) ---

Дельты: ['0.0000', '0.0000', '0.2273', '-0.7273', '0.4545', '-0.0455', '0.0000', '-0.0455', '0.0000']

Ведущий столбец: 4, дельта = 0.4545

Ведущая строка: 3, отношение = 0.0260

=====

ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ 3

=====

	с		-1		-1		-1		-1		-1		0		0		0		0		0		0	
	базис		t1		t2		t3		t4		t5		r1		r2		r3		r4		b			

	t2		0.0000		1.0000		-0.8483		0.5396		0.0000		0.0738		0.0000		-0.0254		-0.0272		0.0213			
	r2		0.0000		0.0000		5.9115		-16.2527		0.0000		-1.0236		1.0000		-0.0319		0.1287		0.0732			
	t1		1.0000		0.0000		0.8684		1.1311		0.0000		-0.0018		0.0000		0.0726		-0.0153		0.0555			
	t5		0.0000		0.0000		1.3799		0.1039		1.0000		-0.0487		0.0000		-0.0032		0.0779		0.0260			
	z/w		0.0000		0.0000		0.3999		0.7745		0.0000		0.0233		0.0000		0.0440		0.0354		0.1027			

РЕЗУЛЬТАТ ДЛЯ ИГРОКА В

Максимальное значение Т = -0.102715

Цена игры v = -9.735632

Оптимальная стратегия В:

y1 = 0.055490

y2 = 0.021251

y3 = 0.000000

y4 = 0.000000

y5 = 0.025974

Выводы

В ходе проделанной работы для игры, заданной матрицей стратегий были найдены оптимальные смешанные стратегии обоих игроков путем сведения матричной игры к задаче ЛП (прямой для одного игрока и двойственной для другого).

Задачи ЛП решены симплекс-методом.

В отчете приведены начальные, промежуточные и конечные симплекс-таблицы.

Цена игры для обоих игроков сошлась и равна 11.

Оптимальная стратегия А:

$$x_1 = 0.023318$$

$$x_2 = 0.000000$$

$$x_3 = 0.043979$$

$$x_4 = 0.035419$$

Оптимальная стратегия Б:

$$y_1 = 0.055490$$

$$y_2 = 0.021251$$

$$y_3 = 0.000000$$

$$y_4 = 0.000000$$

$$y_5 = 0.025974$$

Приложение А

```
import numpy as np

class GamePayoffMatrix:
    def __init__(self):
        self.matrix = [
            [7, 19, 1, 19, 8],
            [7, 18, 5, 2, 6],
            [15, 3, 16, 19, 4],
            [5, 12, 19, 14, 18]
        ]
        self.rows = 4
        self.cols = 5

    def get_matrix(self):
        return self.matrix

    def get_dimensions(self):
        return self.rows, self.cols

class GameSolver:
    def __init__(self, game):
        self.game = game
        self.m = game.rows
        self.n = game.cols
        self.matrix = game.matrix

    def _print_table(self, table, basis, Cb, title, m, n, problem_type):
        rows, cols = table.shape
        print("\n" + "="*90)
        print(title)
        print("="*90)

        if problem_type == 'A':
            print("| C |", end="")
            for _ in range(m): print(" 1 |", end="")
            for _ in range(n): print(" 0 |", end="")
            print(" 0 |")
            print("| Базис |", end="")
            for j in range(m): print(f" u{j+1} |", end="")
            for i in range(n): print(f" s{i+1} |", end="")
            print(" b |")
        else:
            print("| C |", end="")
            for _ in range(m): print(" -1 |", end="")
            for _ in range(n): print(" 0 |", end="")
            print(" 0 |")
            print("| Базис |", end="")
            for j in range(m): print(f" t{j+1} |", end="")
            for i in range(n): print(f" r{i+1} |", end="")
            print(" b |")

        print("-" * (8 + 6 * (m + n + 2)))

        for i in range(rows):
            if i < rows-1:
                print(f"| {basis[i]:>4} |", end="")
            else:
                print("| Z/W |", end="")
            for j in range(cols):
```

```

        val = table[i][j]
        if j == cols-1:
            print(f" {val:8.4f} |", end="")
        else:
            print(f" {val:8.4f} ", end="")
    print()
print("="*90)

def _build_table(self, problem_type):
    m, n = self.m, self.n
    if problem_type == 'A':
        table = np.zeros((n + 1, m + n + 1))
        for i in range(n):
            for j in range(m):
                table[i][j] = -self.matrix[j][i]
            table[i][m + n] = -1
            table[i][m + i] = 1
        for j in range(m):
            table[n][j] = 1
    else:
        table = np.zeros((m + 1, n + m + 1))
        for i in range(m):
            for j in range(n):
                table[i][j] = self.matrix[i][j]
            table[i][n + i] = 1
            table[i][-1] = 1
        for j in range(n):
            table[m][j] = -1
    return table

def _simplex(self, table, problem_type, max_iter=100):
    if problem_type == 'A':
        m, n = self.m, self.n
        basis = [f"s{i+1}" for i in range(n)]
        Cb = [0] * n
        rows, cols = table.shape
    else:
        m, n = self.n, self.m
        basis = [f"r{i+1}" for i in range(self.m)]
        Cb = [0] * self.m
        rows, cols = table.shape

    iteration = 0
    while True:
        if problem_type == 'A':
            neg_rows = [i for i in range(self.n) if table[i][-1] < -1e-8]
        else:
            neg_rows = [i for i in range(self.m) if table[i][-1] < -1e-8]

        if neg_rows:
            pivot_row = min(neg_rows, key=lambda i: table[i][-1])
            pivot_col = -1
            for j in range(cols - 1):
                if table[pivot_row][j] < -1e-8:
                    pivot_col = j
                    break
            if pivot_col == -1:
                break
            print(f"\n--- Итерация {iteration+1} (Фаза 1) ---")
            print(f"Отрицательный b в строке {pivot_row} = {table[pivot_row][-1]:.4f}")
            print(f"Ведущий столбец: {pivot_col}, элемент = {table[pivot_row][pivot_col]:.4f}")
        else:
            deltas = []

```

```

if problem_type == 'A':
    lim = self.n
    for j in range(cols - 1):
        s = sum(Cb[i] * table[i][j] for i in range(lim))
        Cj = 1 if j < self.m else 0
        deltas.append(s - Cj)
    pos = [j for j, d in enumerate(deltas) if d > 1e-8]
    if not pos:
        break
    pivot_col = max(pos, key=lambda j: deltas[j])
    min_ratio = float('inf')
    pivot_row = -1
    for i in range(lim):
        if table[i][pivot_col] > 1e-8:
            ratio = table[i][-1] / table[i][pivot_col]
            if ratio < min_ratio:
                min_ratio = ratio
                pivot_row = i
    if pivot_row == -1:
        break
    print(f"\n--- Итерация {iteration+1} (Фаза 2) ---")
    print(f"Дельты: {[f'{d:.4f}' for d in deltas]}")
    print(f"Ведущий столбец: {pivot_col}, дельта =
{deltas[pivot_col]:.4f}")
    print(f"Ведущая строка: {pivot_row}, отношение = {min_ratio:.4f}")
else: # problem_type == 'B'
    lim = self.m
    for j in range(cols - 1):
        s = sum(Cb[i] * table[i][j] for i in range(lim))
        Cj = -1 if j < self.n else 0
        deltas.append(s - Cj)
    pos = [j for j, d in enumerate(deltas) if d > 1e-8]
    if not pos:
        break
    pivot_col = max(pos, key=lambda j: deltas[j])
    min_ratio = float('inf')
    pivot_row = -1
    for i in range(lim):
        if table[i][pivot_col] > 1e-8:
            ratio = table[i][-1] / table[i][pivot_col]
            if ratio < min_ratio:
                min_ratio = ratio
                pivot_row = i
    if pivot_row == -1:
        break
    print(f"\n--- Итерация {iteration+1} (Фаза 2) ---")
    print(f"Дельты: {[f'{d:.4f}' for d in deltas]}")
    print(f"Ведущий столбец: {pivot_col}, дельта =
{deltas[pivot_col]:.4f}")
    print(f"Ведущая строка: {pivot_row}, отношение = {min_ratio:.4f}")

table, basis, Cb = self._pivot_operation(
    table, pivot_row, pivot_col, basis, Cb, rows, problem_type
)
iteration += 1

if problem_type == 'A':
    self._print_table(table, basis, Cb, f"ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ
{iteration}", self.m, self.n, 'A')
else:
    self._print_table(table, basis, Cb, f"ТАБЛИЦА ПОСЛЕ ИТЕРАЦИИ
{iteration}", self.n, self.m, 'B')
    if iteration >= max_iter:
        break

```

```

        return table, basis, Cb

    def _pivot_operation(self, table, pivot_row, pivot_col, basis, Cb, rows,
        problem_type):
        elem = table[pivot_row][pivot_col]
        table[pivot_row] = table[pivot_row] / elem

        if problem_type == 'A':
            m, n = self.m, self.n
            if pivot_col < m:
                basis[pivot_row] = f"u{pivot_col+1}"
                Cb[pivot_row] = 1
            else:
                basis[pivot_row] = f"s{pivot_col - m + 1}"
                Cb[pivot_row] = 0
        else:
            n_vars = self.n
            if pivot_col < n_vars:
                basis[pivot_row] = f"t{pivot_col+1}"
                Cb[pivot_row] = -1
            else:
                basis[pivot_row] = f"r{pivot_col - n_vars + 1}"
                Cb[pivot_row] = 0

        for i in range(rows):
            if i != pivot_row:
                factor = table[i][pivot_col]
                if abs(factor) > 1e-10:
                    table[i] = table[i] - factor * table[pivot_row]

        return table, basis, Cb

    def _extract_solution(self, table, basis, problem_type):
        if problem_type == 'A':
            m = self.m
            sol = np.zeros(m)
            for j in range(m):
                name = f"u{j+1}"
                if name in basis:
                    idx = basis.index(name)
                    sol[j] = table[idx][-1]
            W = table[-1][-1]
            return sol, W
        else:
            n = self.n
            sol = np.zeros(n)
            for i in range(n):
                name = f"t{i+1}"
                if name in basis:
                    idx = basis.index(name)
                    sol[i] = table[idx][-1]
            T = -table[-1][-1]
            return sol, T

    def _solve(self, problem_type):
        print("\n" + "="*90)
        if problem_type == 'A':
            print("РЕШЕНИЕ ДЛЯ ИГРОКА А (МИНИМИЗАЦИЯ)")
        else:
            print("РЕШЕНИЕ ДЛЯ ИГРОКА В (МАКСИМИЗАЦИЯ)")
        print("="*90)

        table = self._build_table(problem_type)
        if problem_type == 'A':
            self._print_table(table, [f"s{i+1}" for i in range(self.n)], [0]*self.n,
                "НАЧАЛЬНАЯ СИМПЛЕКС-ТАБЛИЦА", self.m, self.n, 'A')

```



```

else:
    self._print_table(table, [f"r{i+1}" for i in range(self.m)], [0]*self.m,
                       "НАЧАЛЬНАЯ СИМПЛЕКС-ТАБЛИЦА", self.n, self.m, 'B')

table, basis, Cb = self._simplex(table, problem_type)
sol, val = self._extract_solution(table, basis, problem_type)

print("\n" + "="*90)
if problem_type == 'A':
    print("РЕЗУЛЬТАТ ДЛЯ ИГРОКА А")
    print(f"Минимальное значение W = {val:.6f}")
    v = 1.0 / val
    print(f"Цена игры v = {v:.6f}")
    print("Оптимальная стратегия А:")
    for i in range(self.m):
        print(f"  x{i+1} = {sol[i]:.6f}")
    return sol, val
else:
    print("РЕЗУЛЬТАТ ДЛЯ ИГРОКА В")
    print(f"Максимальное значение T = {val:.6f}")
    v = 1.0 / val
    print(f"Цена игры v = {v:.6f}")
    print("Оптимальная стратегия В:")
    for i in range(self.n):
        print(f"  y{i+1} = {sol[i]:.6f}")
    return sol, val

def get_strategy_for_minimizer(self): # A
    u, W = self._solve('A')
    return u, W

def get_strategy_for_maximizer(self):
    t, T = self._solve('B')
    v = 1.0 / T
    y = t * v

def main():
    game = GamePayoffMatrix()
    solver = GameSolver(game)

    u, W = solver.get_strategy_for_minimizer()
    v_A = 1.0 / W
    x = u * v_A

    y, v_B = solver.get_strategy_for_maximizer()

    print("=" * 50)
    print("ВАРИАНТ 23")
    print("=" * 50)
    print(f"\nЦена игры (из А): v = {v_A:.6f}")
    print(f"Цена игры (из В): v = {v_B:.6f}\n")
    print("Стратегия А (игрок, минимизирующий):")
    for i in range(game.rows):
        print(f"  x{i+1} = {x[i]:.6f}")
    print("\nСтратегия В (игрок, максимизирующий):")
    for i in range(game.cols):
        print(f"  y{i+1} = {y[i]:.6f}")

```